

FTI-Projekt: Unwettervorhersage mit Open-Meteo und Hopsworks

1. Projektüberblick

Dieses Projekt implementiert eine kleine, aber vollständige MLOps-Architektur im Stil der FTI-Architektur (Feature – Training – Inference):

Feature Pipeline: Wetterdaten aus Open-Meteo laden, Features erstellen und in den Feature Store schreiben.

Training Pipeline: Feature View laden, Modell trainieren und evaluieren.

Inference Pipeline: Neue Wetterdaten verarbeiten und eine Unwetter-Warnung mit einem vortrainierten Modell erzeugen.

Das Ziel ist nicht die Erstellung eines hochoptimierten Wettermodells, sondern die Demonstration eines reproduzierbaren MLOps-Workflows mit:

sauberem Datenfluss,
wiederverwendbaren Features,
modellbasierter Vorhersage,
und klarer Trennung zwischen Datenvorbereitung, Training und Inference.

2. FTI-Architektur im Projekt

Das Projekt folgt dem typischen FTI-Ablauf:

Open-Meteo API



Feature Pipeline



Hopsworks Feature Store



Training Pipeline



Model Registry / model.joblib



Inference Pipeline



2.1 Feature

Die Feature Pipeline sammelt historische und aktuelle Wetterdaten für verschiedene Standorte (z. B. München, Hamburg). Daraus werden Wetter-Features wie Temperatur, Luftdruck, Niederschlag, Wind und zeitliche Trends abgeleitet.

2.2 Training

Die Training Pipeline lädt die Features aus dem Feature Store, erstellt eine Feature View und trainiert ein XGBoost-Klassifikationsmodell. Das Modell lernt, ob ein Wetterereignis als "severe weather" (starkes Unwetter) zu klassifizieren ist.

2.3 Inference

Die Inference Pipeline ruft aktuelle Wetterdaten des gewählten Standorts ab, berechnet die gleichen Features wie im Training und nutzt das gespeicherte Modell, um die Warnung zu berechnen.

3. Datenquelle und Ziel

3.1 Datenquelle

Die Projekt-Daten kommen von der Open-Meteo Weather API:

- historische Daten,
- aktuelle Prognosedaten,
- stündliche Messwerte,
- Standorte: München und Hamburg.

3.2 Zielvariable

Das Ziel ist eine binäre Klassifikation:

- 0 = kein schweres Unwetter
- 1 = schweres Unwetter / Sturmwarnung

Als heuristische Warnkriterien wurden vor allem folgende Signale verwendet:

- starker Wind,
- hohe Niederschlagsmengen über mehrere Stunden,

starker Luftdruckabfall,
Unstetigkeit bzw. Anomalien im Witterungsverlauf.

Diese Logik ist bewusst einfach gehalten, weil das Projekt vor allem die Architektur und den MLOps-Prozess zeigen soll.

4. Feature Pipeline

Die Feature Pipeline ist der erste und wichtigste Bestandteil der FTI-Architektur.

4.1 Datenabruf

In der Funktion `fetch_weather_data(...)` werden über die Open-Meteo API stündliche Wetterdaten für einen Standort geladen. Dabei werden Parameter wie:

Latitude / Longitude,
`past\_days`,
`forecast\_days`,
Zeitzone `Europe/Berlin`,
gewünschte Features,

übergeben.

4.2 Features für die Vorhersage

Aus den Rohdaten werden aggregierte und zeitlich aufgelöste Wetterfeatures berechnet. Beispiele:

`precip\_rolling\_sum\_3h`
`precip\_rolling\_sum\_6h`
`precip\_rolling\_sum\_12h`
`wind\_gust\_max\_3h`
`wind\_gust\_max\_6h`
`wind\_gust\_max\_12h`
`pressure\_mean\_3h`
`pressure\_mean\_6h`
`pressure\_mean\_12h`
`pressure\_change\_3h`
`pressure\_drop\_rate`
`wind\_gust\_anomaly`

``temp_change_3h``

Diese Features sind besonders wichtig, weil sie Dynamiken und kurzfristige Veränderungen abbilden – genau das ist bei Sturm- und Unwetterwarnungen entscheidend.

4.3 Labelbildung

Ein heuristisches Label wird mit Regeln wie diesen erzeugt:

sehr hoher Wind,
starke Regenmengen in kurzen Zeitfenstern,
starker Luftdruckabfall.

Die Spalte `is_severe_weather` wird dadurch als Zielvariable für das Modell erzeugt.

4.4 Finaler Datenrahmen

Die Funktion `build_final_dataframe(...)` stellt den finalen Datensatz zusammen. Dabei werden unter anderem diese Punkte berücksichtigt:

Auswahl relevanter Spalten,
Eindeutige ID ``event_id``,
Zeitstempel ``event_time``,
Behandlung von NaN-Werten im Bereich der Rolling Features.

4.5 Feature Store

Der fertige Datensatz wird in eine Hopsworks Feature Group hochgeladen. Die Feature Group hat folgende Funktion:

Wiederverwendbarkeit für Training und Inference,
zentrale Speicherung der Features,
Versionierung und Event Time,
einfache Nachnutzung in anderen Pipelines.

5. Training Pipeline

Die Training Pipeline nutzt die im Feature Store gespeicherten Daten.

5.1 Laden der Feature Group

Das Notebook lädt die Feature Group `weather_features_batch` aus Hopsworks und prüft, ob die benötigten Spalten vorhanden sind.

5.2 Erstellung einer Feature View

Es wird eine Feature View `severe_weather_fv` erzeugt, die nur die relevanten Features plus das Label enthält. Dadurch kann das Modell zuverlässig mit den gleichen Feature-Spalten arbeiten, die auch bei der Inference verwendet werden.

5.3 Train/Test-Split

Die Daten werden mit `train_test_split(...)` in Trainings- und Testdaten aufgeteilt. Dabei wird `stratify=y` verwendet, damit beide Klassen in beiden Datensätzen vertreten sind.

5.4 Modell

Das verwendete Modell ist ein `XGBClassifier` aus XGBoost, ein starkes Modell für tabellarische Daten.

Wichtige Designentscheidungen:

- ``class_weight`` zur Ausgleichung der Klassenverteilung,
- ``scale_pos_weight`` zur stärkeren Berücksichtigung der Unwetterklasse,
- ``random_state=42`` für Reproduzierbarkeit.

5.5 Evaluation

Die Evaluation erfolgt mit klaren Klassifizierungsmetriken:

- Precision,
- Recall,
- F1-Score,
- ROC-AUC,
- Confusion Matrix.

Die laut Projekt hinterlegten Metriken zeigen:

- F1-Score: 1.0
- ROC-AUC: 1.0
- Train samples: 1172
- Test samples: 294

Das zeigt, dass das Modell auf dem erzeugten Datensatz sehr stark performt. Wichtig ist aber: Das ist primär eine Architektur- und MLOps-Demo, nicht eine produktionsreife Wettervorhersage.

5.6 Modell-Storage

Das Modell wird lokal als `model.joblib` gespeichert und mit den Metriken in der Model Registry bzw. im Projektordner abgelegt.

6. Inference Pipeline

Die Inference Pipeline ist der dritte Schritt der FTI-Architektur und bildet den echten Echtzeit-Teil.

6.1 Live-Daten laden

Es werden aktuelle Wetterdaten für einen Ort über Open-Meteo abgerufen. Dabei werden ebenfalls eine historische Vergangenheit und ein kurzer Forecast-Bereich berücksichtigt.

6.2 Feature-Vektor erzeugen

Aus den aktuellen Wetterdaten werden dieselben Features berechnet, die im Training verwendet wurden. Dadurch bleibt die Feature-Logik konsistent:

- gleiche Spalten,
- gleiche Berechnungsregeln,
- gleiche Proxy-Logik für Unwetter.

6.3 Modell laden

Das gespeicherte Modell wird aus der Model Registry bzw. dem lokalen Modellordner geladen. Anschließend wird ein einzelner Feature-Vektor für die Vorhersage an das Modell übergeben.

6.4 Prediction

Die Funktion `run_realtime_prediction(...)`:

- prüft, ob alle erwarteten Feature-Spalten vorhanden sind,
- baut einen DataFrame mit genau den richtigen Spalten,
- ruft ``model.predict_proba(...)`` auf,
- berechnet die Wahrscheinlichkeit für Unwetter,
- setzt eine Warnschwelle, z. B. 0.3.

Ein einfacher Zufallswert wird nicht verwendet; die Reihenfolge des Workflows ist identisch mit einer realistischen MLOps-Pipeline:

- Daten abrufen,

Features berechnen,
Modell laden,
Score berechnen,
Warnung ausgeben.

7. Warum diese Architektur sinnvoll ist

Die FTI-Architektur ist hier besonders passend, weil sie drei Dinge sauber trennt:

7.1 Feature Engineering

Die Datenvorbereitung und Feature-Erzeugung sitzt in einer eigenen Pipeline. Das ist wichtig, weil Features:

wiederverwendbar sein sollen,
konsistent zwischen Training und Inference sein müssen,
zentral gepflegt werden sollten.

7.2 Training

Das Training ist isoliert und kann wiederholt ausgeführt werden. So kann das Modell optimiert, verglichen und verwaltet werden.

7.3 Inference

Die Vorhersage läuft separat von der Trainingslogik. Dadurch kann der Live-Score jederzeit auf neuem Wetterdatenmaterial berechnet werden.

Diese Trennung ist eine Kernidee von MLOps und genau das, was dieses Projekt demonstriert.

8. Stärken und Grenzen des Projekts

Stärken

klare FTI-Struktur,
Nutzung eines echten Wetter-APIs,
Feature Store Integration mit Hopsworks,
Modell-Registry-Workflow,
saubere Trennung der Verantwortlichkeiten,

nachvollziehbare Pipeline-Architektur.

Grenzen

Das Label ist heuristisch und nicht aus echten Warnmeldungen abgeleitet.

Die Datenbasis ist klein und bewusst einfach.

Es gibt keine echte Produkt-Deployment-Umgebung,

die Vorhersage ist eher ein Architektur- und Lernprojekt als ein operationelles Wetter-Produkt.

9. Fazit

Das Projekt zeigt eine vollständige und verständliche FTI-Architektur für eine kleine KI-Anwendung im Wetterbereich:

Feature Pipeline sammelt und transformiert Daten.

Training Pipeline nutzt das Feature Store und trainiert ein Modell.

Inference Pipeline nutzt aktuelle Daten zur Warnung.

Damit entspricht das Projekt sehr gut den gewünschten Lernzielen einer MLOps-Projektarbeit: Datenfluss, Feature Engineering, Modelltraining, Wiederverwendung von Features und klare Trennung der Pipeline-Schritte.

10. Wichtige Projektdateien

Feature Pipeline: `Feature Pipeline/feature\_pipeline.ipynb`

Trainings Pipeline: `Trainings Pipeline/trainings\_pipeline.ipynb`

Inference Pipeline: `Inference Pipeline/inference\_pipeline.ipynb`

Modell: `Trainings Pipeline/severe\_weather\_model/model.joblib`

Metriken: `Trainings Pipeline/severe\_weather\_model/metrics.json`

README: `README.md`

11. Kurzfazit in einem Satz

Dieses Projekt zeigt, wie man mit Open-Meteo, Hopsworks und einem XGBoost-Modell eine moderne FTI-Architektur für eine Unwettervorhersage aufbaut – mit Fokus auf Wiederverwendbarkeit, MLOps-Logik und verständlicher Pipeline-Struktur.